

## Q1

写一个 C 表达式，在下列描述的条件下产生 1，其他情况产生 0，假设 X 是 int 类型。代码中不能使用 == 或 != 进行测试。

- x 的任何位都等于 1；
- x 的任何位都等于 0；
- x 的最低有效字节中的位都等于 1；
- x 的最高有效字节中的位都等于 1；

## Answer1

```
// x == a 等价于 !(x^a)
int test(int n)
{
    // 任何位都为1
    if (!(~n)) return 1;

    // 任何位都为0
    if (!n) return 1;

    // 最低有效字节都为1
    if (!(n & 0xff) ^ 0xff)
        return 1;

    // 最高有效字节都为1
    if (!((n >> 24) & 0xff) ^ 0xff)
        return 1;
    // if (!(n & 0xff000000) ^ 0xff000000) 亦可

    return 0;
}
```

## Q2

int 为 32 位，float 和 double 分别是 32 位和 64 位 IEEE 格式

```
Int x = random();
```

```
Int y = random();
```

```
Int z = random();
```

```
Double dx = (double)x;
```

```
Double dy = (double)y;
```

```
Double dz = (double)z;
```

对于下面的每个 C 表达式，判断是否恒为 1。如果是请说明原理，如果不是请举出反例。

A. `(float)x == (float)dx`

B. `dx-dy == (double)(x-y)`

C. `(dx+dy)+dz == dx+(dy+dz)`

D. `(dx*dy)*dz == dx*(dy*dz)`

E. `dx/dx == dz/dz`

## Answer2

**A. `(float)x == (float)dx`**

正确

x 为 int，转 float 会丢失精度

x 强转为 double 不会丢失精度，但是再转为 float 会丢失，但还是相当于从 int 转 float。

因此就算是丢失，也是丢失相同的，最后两个值还是会相等。

**B. `dx-dy == (double)(x-y)`**

错误

double 可以完全表示 int 而不丢失精度，且溢出的 **`2*INT_MAX`** 也能放得下，因此 `dx-dy` 算出来的结果就是真实的 `x-y` 的结果，不过表示为了 double。

而 `x-y` 本身运算时候还是 int，考虑可能出现**溢出**，也就是说 **`x-y > INT_MAX`**，会变为负数，因此和我们现实中算出来的 `x-y` 再转 double 是不相通的。

给出的反例

```
int main()
```

```

{
    // 会造成溢出的 x-y
    int x = 2147483647;
    int y = -2147483648;
    double dx = (double)x;
    double dy = (double)y;

    printf("x: %.20f\n", dx);
    printf("y: %.20f\n", dy);
    printf("dx - dy: %.20f\n", dx - dy);
    printf("double(x - y): %.20f\n", (double)(x - y));

    if ((dx - dy) == (double)(x - y))
        printf("Equal\n");
    else
        printf("Not Equal\n");

    return 0;
}

```

**C.  $(dx+dy)+dz == dx+(dy+dz)$**

正确

注意到 **dx**、**dy**、**dz** 都是由 **int** 转为 **double** 的，而非随意构造的，且就算是 3 个 **T\_MIN/T\_MAX** 相加减也不会溢出 **double** 的表示。故在整数下（不过是添了.0000 且不会溢出），因此结合律还是存在的。

**D.  $(dx*dy)*dz == dx*(dy*dz)$**

错误

只需要构造出 **double** 不能精确表示的整数即可，边界情况则是考虑用 **INT\_MAX** 相乘，**z** 随便赋一个大数就会造成误差。参考 **float** 在 16777216 之后就会丢失 **int** 精度。

给出的反例

```

#include <iostream>
#include <cmath>

int main()
{
    int x = 2147483647;
    int y = 2147483647;
    int z = 214748336;
    double dx = (double)x;
    double dy = (double)y;
    double dz = (double)z;

    // 计算 (dx * dy) * dz 和 dx * (dy * dz)
    double result1 = (dx * dy) * dz;
    double result2 = dx * (dy * dz);

    // 输出结果
    std::cout << "(dx * dy) * dz = " << result1 << std::endl;
    std::cout << "dx * (dy * dz) = " << result2 << std::endl;

    // 也可以换成其他的, 1e-2这种。不过给出的这个例子已经有很大误差了
    if (fabs(result1 - result2) - 0 < 1)
    {
        std::cout << "The expressions are NOT equal!" << std::endl;
    }
    else
    {
        std::cout << "The expressions are equal!" << std::endl;
    }

    return 0;
}

```

E.  $dx/dx == dz/dz$

错误

考虑到除数不能为 0，取 $x=0$ ， $z=1$ 就是反例了。

### Q3

- 编写如下函数，求浮点数  $f$  的绝对值 $|f|$ 。如果  $f$  是 NaN，那么应该直接返回  $f$ （注意 NaN 不要对  $f$  做任何修改）。
- 其中 float\_bits 等价于 unsigned，是 float 数字的二进制形式

### Answer3

- 考虑到 NAN 的 float 表示是，exp 全 1，frac 为非 0，特判出来直接返回。
- 其他情况只要把符号为置 0 即可。

```
/* Compute |f|. If f is NaN, then return f. */
typedef unsigned float_bits;
float_bits float_absval(float_bits f)
{
    float_bits sign = f & 0x80000000;
    float_bits exp = f & 0x7f800000;
    float_bits frac = f & 0x007fffff;

    if (exp == 0x7f800000 && frac != 0)
        return f;

    return f & 0x7fffffff;
}
```

### Q4

实现如下函数，对于浮点数  $f$ ，计算  $2.0*f$ 。如果  $f$  是 NaN，你的函数应该简单返回  $f$ 。

### Answer4

datalab 已有此题，不再过多叙述，注释已经说明思路了。

```
/* Compute 2*f. If f is NaN, return f. */
float_bits float_twice(float_bits f){
    float_bits sign = f&0x80000000;
    float_bits exp = (f>>23)&0xff;
    float_bits frac = f&0x7fffff;

    // 处理NaN, 但如果是INF也可以直接返回
    if(exp==0xff) return f;

    exp++;

    // 处理溢出, 这时候尾数就不需要了
    if (exp==0xff)
    {
        return sign|0x7f800000;
    }

    // 考虑规格和非规格化数
    if(exp==1){
        frac<<=1;
        return sign|frac; // 还是非规格化数, exp不需要++
    }

    return sign|(exp<<23)|frac;
}
```